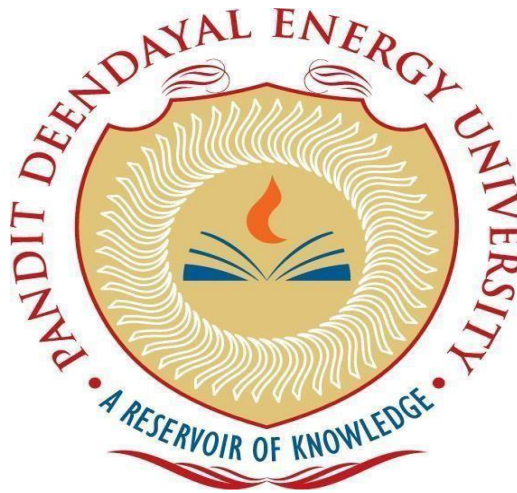


PROJECT REPORT

PANDIT DEENDAYAL ENERGY UNIVERSITY, GANDHINAGAR
DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING



Course Title : Electronics System Design Lab

Project Title : Bearing Fault Detection System
(AI-Powered Bearing Monitoring using ESP32 & MPU-6050)

Course Code : 20ECE312P

Semester : 6th

Academic Year : 2025-2026

Faculty Name: Dr. Abhishek Ray

Date of Submission: 17th April 2026

Submitted by:

Diya Arvadia (23BEC094)

Shweta Sharma (23BEC095)

Table of Contents

Sr No.	Topic	Page No.
1.	Abstract	5
2.	Introduction 2.1 Background 2.2 Problem Statement 2.3 Objectives 2.4 Importance of Predictive Maintenance	6
3.	Literature Survey / Related Work 3.1 Classical Vibration Analysis Approaches 3.2 Machine Learning for Bearing Fault Detection 3.3 Evolution to TinyML and Edge Deployment	8
4.	System Overview 4.1 Overall Architecture 4.2 Data Flow	10
5.	Hardware Design 5.1 ESP32-WROOM-32 Microcontroller 5.2 MPU-6050 Inertial Measurement Unit 5.3 L298N Motor Driver Module 5.4 DC Motor with Unbalanced Load 5.5 Output Peripherals	12
6.	Software and Model Design	16
7.	Working Principle	19
8.	Circuit Design	20
9.	Results and Testing	23
10.	Applications	26
11.	Future Scope	28
12.	Conclusion	30
13.	References	31
14.	Appendices	33

1. Abstract

This report presents the design, implementation, and evaluation of an edge artificial intelligence system for real-time bearing fault detection in rotating machinery. Bearing failures represent one of the most prevalent causes of unplanned downtime in industrial equipment, accounting for a significant proportion of motor and mechanical system failures globally. Conventional monitoring approaches rely on periodic manual inspection or expensive vibration analysis equipment, both of which are ill-suited to the demands of modern predictive maintenance frameworks.

The proposed system addresses these shortcomings by deploying a one-dimensional Convolutional Neural Network (1D-CNN) model — trained on the widely benchmarked Case Western Reserve University (CWRU) bearing dataset — directly onto an ESP32 microcontroller. The model, quantised to 8-bit integer (INT8) format using TensorFlow Lite Micro, occupies approximately 18 kilobytes of flash memory and performs inference within 5 to 15 milliseconds per 128-sample window, enabling continuous real-time classification at a practical sampling rate.

The system classifies three bearing conditions — Normal, Inner Race Fault, and Outer Race Fault — with an accuracy of 99.8 percent on noise-augmented test data, demonstrating robust generalisation under realistic sensor noise conditions. Hardware components include the MPU-6050 three-axis accelerometer, an L298N motor driver module, a DC motor with an unbalanced load for fault simulation, tri-colour LED indicators, an active buzzer for audible alerts, and an optional OLED display for on-device visualisation. The complete system is estimated to cost under Rs. 1,300, representing a highly cost-effective alternative to commercial bearing monitoring solutions. The work demonstrates the viability of TinyML for predictive maintenance at the edge in resource-constrained embedded environments.

2. Introduction

2.1 Background

Rotating machinery forms the backbone of virtually every industrial and manufacturing operation — from electric motors and pumps to compressors, conveyors, and turbines. Within these systems, rolling element bearings serve a critical mechanical function: they support rotating shafts, reduce friction between moving components, and transmit loads across mechanical interfaces. Despite their seemingly simple design, bearings are subject to cyclic stresses, fatigue, contamination, and misalignment that inevitably lead to progressive degradation and eventual failure.

According to studies published by the Electric Power Research Institute (EPRI) and the SKF Group, bearing failures account for approximately 40 to 50 percent of all motor failures in industrial environments. The consequences of undetected bearing degradation extend far beyond the cost of component replacement — they encompass lost production, emergency maintenance labour, secondary equipment damage, and in safety-critical applications, risk to human life. The annual economic loss attributable to unplanned machinery downtime in manufacturing industries is estimated to exceed USD 50 billion globally.

Vibration analysis has long been the predominant technique for bearing condition monitoring. A healthy bearing generates a characteristic vibration signature — typically low-amplitude, quasi-random noise — while a defective bearing produces periodic impulsive signals at frequencies determined by its geometry and rotational speed. These fault characteristic frequencies, known as the Ball Pass Frequency Inner race (BPFI), Ball Pass Frequency Outer race (BPFO), and Ball Spin Frequency (BSF), have been extensively studied in the literature and form the theoretical foundation for frequency-domain diagnostic approaches.

2.2 Problem Statement

Despite decades of research in vibration-based bearing diagnostics, the widespread deployment of continuous bearing monitoring systems in small and medium enterprises (SMEs) remains limited. Traditional vibration analysis platforms require expensive spectrum analysers, trained signal processing engineers, and significant infrastructure investment — barriers that are prohibitive for the vast majority of industrial operators. Furthermore, manual inspection-based maintenance regimes are fundamentally reactive: they detect failures only after they have progressed to a point of observable physical degradation, often too late to prevent catastrophic failure.

Recent advances in machine learning have demonstrated that data-driven models can learn

fault signatures from raw vibration data with a level of accuracy and generalisation that rivals or exceeds traditional signal processing methods. However, the deployment of these models has historically been confined to cloud or edge-server environments, requiring continuous data transmission, network connectivity, and substantial computational resources. This architecture introduces latency, communication overhead, privacy concerns, and points of failure that are incompatible with the reliability requirements of industrial monitoring systems.

The emergence of TinyML — the discipline of deploying machine learning inference on microcontrollers and other ultra-low-power embedded systems — presents a compelling solution to these challenges. By performing inference locally on the sensor node itself, TinyML-based systems can deliver real-time fault detection with minimal latency, zero network dependency, and at a fraction of the cost of conventional solutions.

2.3 Objectives

The primary objectives of this project are as follows:

- To develop a 1D-CNN model trained on the CWRU bearing dataset capable of classifying three bearing conditions: Normal, Inner Race Fault, and Outer Race Fault.
- To apply INT8 post-training quantisation to the trained model and deploy it using TensorFlow Lite Micro on the ESP32 microcontroller.
- To design and assemble a complete hardware prototype integrating the MPU-6050 accelerometer, L298N motor driver, DC motor, and output peripherals.
- To validate the system's classification accuracy under both clean and noise-augmented conditions, and to demonstrate real-time fault detection on physical hardware.
- To evaluate the system's suitability for deployment in resource-constrained industrial environments based on model size, inference latency, and component cost.

2.4 Importance of Predictive Maintenance

Predictive maintenance (PdM) represents a fundamental shift in industrial asset management philosophy — from reactive repair and scheduled preventive maintenance to condition-based intervention triggered by real-time data. The economic case for PdM is compelling: studies by Deloitte indicate that predictive maintenance can reduce maintenance costs by 25 percent, increase equipment uptime by up to 20 percent, and extend asset life by years in high-utilisation applications. For rotating machinery specifically, the ability to detect bearing degradation weeks before failure enables planned replacement during scheduled shutdowns, eliminating the far greater cost of emergency intervention.

3. Literature Survey / Related Work

3.1 Classical Vibration Analysis Approaches

The scientific investigation of bearing fault detection through vibration analysis dates to the foundational work of Randall and Antoni, whose comprehensive survey of rolling element bearing diagnostics established the theoretical framework that continues to underpin the field. Early methods relied on time-domain statistical indicators — root mean square (RMS) amplitude, peak value, kurtosis, and crest factor — applied to raw accelerometer signals. Of these, kurtosis proved particularly discriminating: while a healthy bearing exhibits near-Gaussian vibration statistics with a kurtosis of approximately 3.0, the impulsive shocks generated by a defective bearing drive the kurtosis to values of 5.0 or above, providing a scalar fault indicator requiring no frequency decomposition.

Frequency-domain methods, particularly the envelope analysis technique developed by McFadden and Smith, extended diagnostic capability by extracting fault characteristic frequencies from demodulated vibration signals. The high-frequency resonance technique, wherein the vibration signal is band-pass filtered around structural resonances before envelope detection, became the industry standard for rolling element bearing diagnostics and was incorporated into commercial monitoring systems throughout the 1990s and 2000s. Subsequent refinements, including the Kurtogram for automated resonance band selection proposed by Antoni, further improved robustness against background noise.

3.2 Machine Learning for Bearing Fault Detection

The application of supervised machine learning to bearing fault detection gained momentum following the public release of the Case Western Reserve University (CWRU) bearing dataset in 2000. This benchmark dataset, comprising vibration signals recorded from a laboratory test rig under controlled fault conditions, enabled systematic comparison of diagnostic algorithms across research groups and became the most widely cited resource in the bearing fault detection literature, with over 10,000 publications referencing it as of 2024.

Early machine learning approaches to CWRU-based fault detection employed support vector machines (SVM) operating on hand-crafted feature vectors comprising time-domain and frequency-domain statistical descriptors. Lei et al. demonstrated SVM classifiers achieving over 95 percent accuracy on three-class bearing fault classification, while Sugumaran et al. explored the combination of feature selection techniques with SVM to reduce computational overhead without sacrificing diagnostic accuracy. However, the reliance on domain expertise for feature engineering represented a fundamental limitation of these approaches, as the optimal feature set is inherently signal-dependent and may not generalise across machines or

operating conditions.

Deep learning methods, particularly Convolutional Neural Networks (CNN), emerged as a powerful alternative by learning fault-discriminative features directly from raw or minimally preprocessed vibration signals. Zhang et al. proposed a one-dimensional CNN architecture that accepted raw time-series vibration data as input, demonstrating that end-to-end deep learning models could match or exceed the diagnostic accuracy of feature-engineered SVM approaches while requiring significantly less domain expertise. Subsequent work by Wen et al. extended the CNN approach to two-dimensional representations by converting vibration signals into greyscale images, achieving state-of-the-art accuracy on multiple benchmark datasets.

3.3 Evolution to TinyML and Edge Deployment

The deployment of trained neural network models on microcontrollers — collectively termed TinyML — represents the most recent phase in the evolution of machine learning-based bearing diagnostics. Warden and Situnayake's seminal textbook on TinyML articulated the technical framework for model compression and deployment on Arm Cortex-M class processors, establishing quantisation, pruning, and knowledge distillation as the primary techniques for reducing model footprint to the kilobyte scale without unacceptable accuracy degradation.

In the bearing fault detection domain, Janssens et al. demonstrated the feasibility of deploying a convolutional feature extractor on an ARM Cortex-M4 microcontroller embedded in a wireless sensor node, achieving real-time fault classification with latencies below 10 milliseconds. More recently, the availability of the TensorFlow Lite Micro framework and the EloquentTinyML library for Arduino-compatible platforms has significantly lowered the technical barrier to edge ML deployment, enabling researchers and engineers without deep embedded systems expertise to deploy quantised models on commodity microcontrollers such as the ESP32.

4. System Overview

4.1 Overall Architecture

The proposed bearing fault detection system adopts a layered architecture comprising four functional subsystems: signal acquisition, signal processing and inference, output and alerting, and motor control. These subsystems are orchestrated by the ESP32-WROOM-32 microcontroller, which serves as the central processing unit for all real-time operations.

At the sensing layer, the MPU-6050 inertial measurement unit is physically mounted on the body of a DC motor and communicates with the ESP32 via the Inter-Integrated Circuit (I2C) serial interface. The motor itself is driven by an L298N H-bridge motor driver, which accepts pulse-width modulated (PWM) control signals from the ESP32 and supplies the requisite current from a separate 9V power supply. An analogue potentiometer connected to one of the ESP32's analogue-to-digital converter (ADC) input pins provides manual control over the motor's rotational speed, and consequently, the intensity of the vibration presented to the accelerometer.

The inference subsystem operates continuously in the main processing loop. The ESP32 samples the Z-axis acceleration output of the MPU-6050 at approximately 1000 samples per second, accumulating readings into a 128-sample window buffer. Upon completion of each window, the buffered signal undergoes normalisation, INT8 quantisation, and presentation to the TFLite Micro interpreter, which executes the 1D-CNN model and produces a three-element softmax probability vector. The class with the highest probability is selected as the fault prediction, and the associated confidence score is compared against a configurable threshold.

The output subsystem translates the classification result into human-observable indicators: a tri-colour LED array (green for Normal, yellow for Inner Race Fault, red for Outer Race Fault), an active piezoelectric buzzer triggered on confirmed fault detection above the confidence threshold, and an OLED display presenting the fault class label and confidence percentage.

4.2 Data Flow

The end-to-end data flow from physical vibration to fault alert proceeds through seven sequential stages. In Stage 1, the rotating DC motor with an eccentrically mounted mass generates mechanical vibration that propagates through the motor housing to the rigidly mounted MPU-6050 sensor. In Stage 2, the MPU-6050 digitises the analogue acceleration signal at 12-bit resolution across a configurable measurement range, transferring readings to the ESP32 via I2C at the configured output data rate. Stage 3 involves the accumulation of 128 sequential accelerometer readings into a circular buffer, representing approximately 128 milliseconds of vibration data at the 1 kHz sampling rate. Stage 4 normalises the buffered

signal to the range $[-1, 1]$ by dividing each sample by the maximum absolute value in the window, ensuring compatibility with the model's training data distribution. Stage 5 quantises the normalised floating-point values to signed 8-bit integers using the scale factor and zero-point determined during post-training quantisation calibration. Stage 6 executes the 1D-CNN inference using the TFLite Micro interpreter, producing probability scores for each of the three fault classes. Stage 7 decodes the inference output, updates the LED indicators and buzzer state, and transmits diagnostic information via the UART serial interface for monitoring and logging purposes.

5. Hardware Design

5.1 ESP32-WROOM-32 Microcontroller

The ESP32-WROOM-32 serves as the central processing unit of the proposed system. Manufactured by Espressif Systems, the ESP32 integrates a dual-core Xtensa LX6 processor operating at up to 240 MHz, 520 kilobytes of static random-access memory (SRAM), 448 kilobytes of read-only memory (ROM), and interfaces to 4 megabytes of external SPI flash memory. The processor's dual-core architecture allows concurrent execution of sensor sampling on one core while the second core handles inference and communication tasks, eliminating scheduling conflicts that could introduce timing jitter in the acquisition pipeline.

For the purposes of TFLite Micro inference, the ESP32's SRAM provides the tensor arena — the contiguous memory region allocated for intermediate activation tensors during model execution. The 1D-CNN model employed in this project requires a tensor arena of approximately 20 kilobytes, well within the available SRAM budget. The flash memory stores the quantised model weights (approximately 18 kilobytes), the compiled application firmware, and partition table metadata. The Huge APP partition scheme is employed to allocate the maximum flash region to the application binary, accommodating the TFLite runtime library.

The ESP32 exposes 34 programmable GPIO pins, of which the system utilises: GPIO 21 and GPIO 22 for I2C communication with the MPU-6050; GPIO 25, 26, and 27 for motor driver control (ENA, IN1, IN2); GPIO 33, 14, and 12 for LED drive; GPIO 32 for buzzer control via a BC547 NPN transistor; and GPIO 34 for analogue potentiometer input. The device is powered via its USB Micro connector from a laptop or USB power bank during development and demonstration.

5.2 MPU-6050 Inertial Measurement Unit

The MPU-6050, manufactured by InvenSense (now TDK), is a six-axis inertial measurement unit integrating a three-axis MEMS accelerometer and a three-axis MEMS gyroscope on a single die. For this application, only the accelerometer function is utilised, specifically the Z-axis output aligned with the radial direction of the motor bearing. The device communicates with the host microcontroller via I2C at clock rates of up to 400 kHz (Fast Mode), enabling data transfer rates sufficient for the 1 kHz sampling requirement of the signal acquisition pipeline.

The MPU-6050's accelerometer provides selectable full-scale measurement ranges of plus or minus 2g, 4g, 8g, or 16g, with 16-bit ADC resolution in each range. For the motor vibration characterisation task, the plus or minus 4g range is appropriate, providing sufficient headroom for the peak accelerations generated by the unbalanced motor (measured at approximately

3.6g at maximum speed) while maintaining adequate resolution for the low-amplitude Normal condition signals. The device incorporates an onboard Digital Motion Processor (DMP) and a 1024-byte FIFO buffer, though neither feature is utilised in this implementation, which opts for direct register polling via the MPU6050_light Arduino library.

The MPU-6050 breakout module is physically secured to the motor housing using two-component epoxy adhesive (Araldite), ensuring a rigid mechanical coupling that faithfully transmits bearing vibration to the sensor without resonance or damping effects introduced by loose mounting. The I2C address of the device is configured to 0x68 by connecting the AD0 address selection pin to the system ground rail.

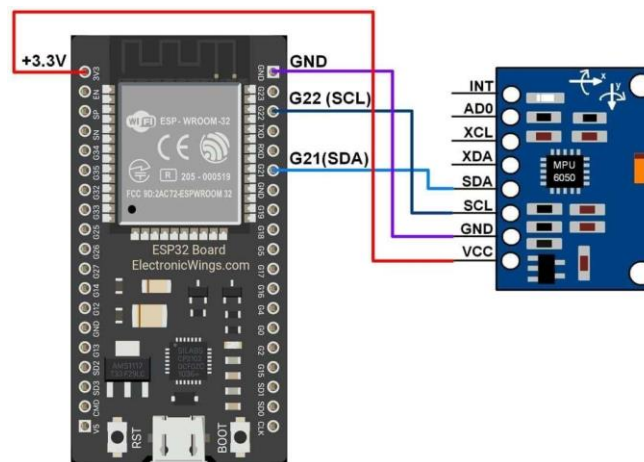


Figure 5.1: ESP32 interfaced with MPU-6050 accelerometer

5.3 L298N Motor Driver Module

The L298N is a monolithic integrated H-bridge driver circuit capable of driving two independent DC motors or a single bipolar stepper motor. In this application, one H-bridge channel drives the DC motor used as the vibration source. The L298N module accepts a logic-level enable signal (ENA) and two direction control signals (IN1, IN2) from the ESP32, with the motor speed determined by the duty cycle of the PWM signal applied to the ENA input. The device's power stage accepts supply voltages from 5V to 35V and delivers continuous output currents of up to 2 amperes per channel, with transient peak currents of 3 amperes.

A critical design consideration in the motor driver interface is the requirement for a shared ground reference between the ESP32 and the L298N module. Since the ESP32 GPIO signals reference the ESP32 ground, and the L298N logic inputs reference its own ground, failure to connect these two grounds together results in an undefined common-mode voltage that prevents reliable signal transfer. All ground connections — ESP32 GND, L298N GND, and battery negative terminal — are therefore interconnected at a common ground rail on the

prototype breadboard.

5.4 DC Motor with Unbalanced Load

A small hobby DC motor is employed as both the mechanical vibration source and the system under test. To simulate bearing fault conditions, a small mass (a one-rupee coin, mass approximately 3.7 grams) is affixed to the motor shaft at an eccentric radial offset using epoxy adhesive. This creates a rotating unbalance that generates centrifugal force proportional to the product of the unbalanced mass, the eccentricity, and the square of the angular velocity — physically analogous to the periodic impulsive forces generated by a defective bearing contacting a race defect during each revolution. By varying the motor speed via the potentiometer, the vibration intensity presented to the MPU-6050 can be modulated to simulate fault conditions of varying severity, providing an observable demonstration of the classification system's dynamic response.

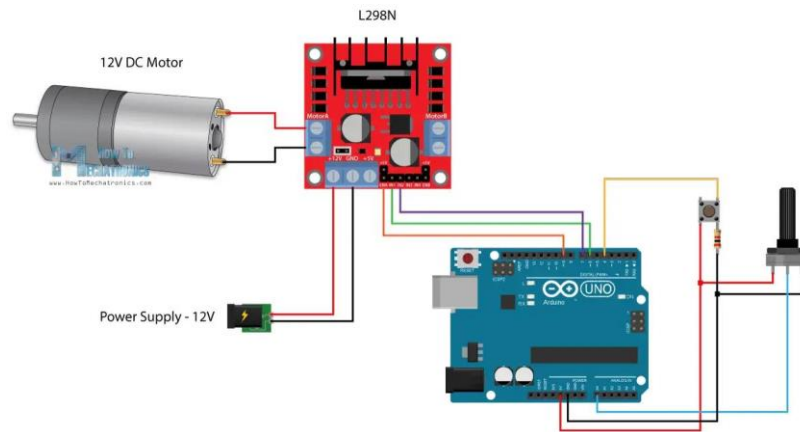


Figure 5.2: DC motor with L298N driver used for vibration generation

5.5 Output Peripherals

Three five-millimetre light-emitting diodes — green, yellow, and red — provide immediate visual indication of the current fault classification. Each LED is connected in series with a 220-ohm current-limiting resistor between the corresponding ESP32 GPIO output and the common ground rail. The resistor value is selected to limit the LED forward current to approximately 15 milliamperes at 3.3V GPIO output voltage, within the safe operating range for the ESP32 GPIO source current capability of 40 milliamperes per pin. A five-volt active buzzer provides audible fault alerting. Since the buzzer draws approximately 30 to 50 milliamperes of operating current — exceeding the safe GPIO source capability — it is driven through a BC547 NPN bipolar transistor configured as a common-emitter switch. The ESP32 GPIO output drives the transistor base through a 1-kiloohm base resistor, while the buzzer is connected between the VIN (5V USB) supply and the transistor collector, with the emitter referenced to system ground.

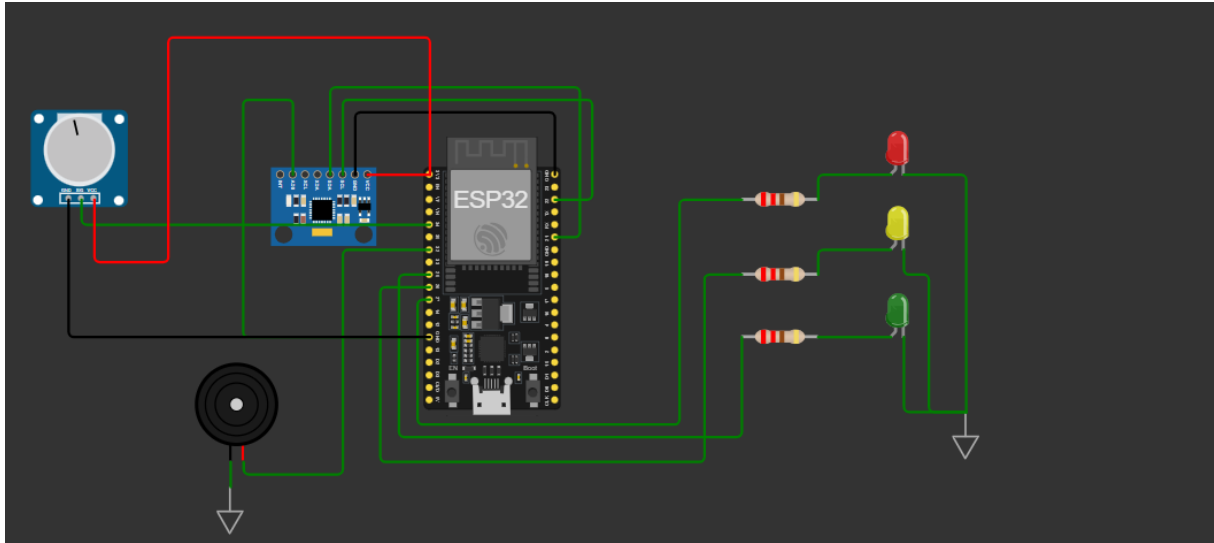


Figure 5.3: Complete hardware prototype of the system

Hardware Components Table

Component	Specification	Purpose
ESP32	Dual-core, 240 MHz	Main controller
MPU-6050	3-axis accelerometer	Vibration sensing
L298N	Motor driver	Motor control
DC Motor	9V motor	Fault simulation
LEDs	RGB	Status indication
Buzzer	5V active	Alert system

6. Software and Model Design

6.1 Dataset Description

The Case Western Reserve University (CWRU) Bearing Dataset, maintained by the Bearing Data Centre of Case Western Reserve University, Ohio, USA, constitutes the primary training and evaluation resource for the fault detection model. The dataset was collected from a purpose-built motor-bearing test rig comprising a 2-horsepower induction motor, a torque transducer, and a dynamometer load. Bearing faults of three diameters (0.007, 0.014, and 0.021 inches) were introduced at three locations — inner race, outer race, and rolling element — using electro-discharge machining (EDM), a process that creates precisely dimensioned surface defects without inducing secondary damage.

For this project, three files are utilised from the CWRU repository, all corresponding to the 0.007-inch fault diameter at zero horsepower load and approximately 1797 revolutions per minute shaft speed: Normal_0hp.mat (243,938 samples, 20.3 seconds duration), IR007_0.mat representing inner race fault (121,265 samples, 10.1 seconds), and OR007_6_0.mat representing outer race fault at the six o'clock load position (121,991 samples, 10.2 seconds). All files contain vibration data sampled at 12,000 samples per second using a research-grade accelerometer mounted at the drive end (DE) bearing location. The DE channel is selected for model training as it is positioned closest to the test bearing and exhibits the highest signal-to-noise ratio for fault-related vibration components.

6.2 Data Preprocessing and Feature Extraction

Raw vibration signals are segmented into non-overlapping and overlapping windows of 128 samples, corresponding to approximately 10.7 milliseconds of data at the 12 kHz sampling rate. A window overlap of 50 percent is applied during training to increase the effective dataset size through data augmentation, yielding approximately 3,810 windows from the Normal class, 947 from the Inner Race class, and 953 from the Outer Race class, for a total of 5,710 training windows before noise augmentation. Each window is independently normalised to the range $[-1, 1]$ by dividing by the maximum absolute sample value within the window, ensuring that the model's input distribution is consistent regardless of the absolute vibration magnitude.

While the 1D-CNN model learns its own internal feature representations from the normalised time-series input, the rule-based fallback classifier and the Serial Monitor output report three classical vibration features computed from each window: Root Mean Square (RMS) amplitude, which quantifies the overall vibration energy and correlates strongly with fault severity; Kurtosis, the fourth standardised statistical moment of the signal distribution, which measures the impulsiveness of the signal and is the most sensitive single indicator of incipient bearing

faults, typically rising from approximately 3.0 for Gaussian (healthy) signals to values of 5 to 8 for faulty bearings; and Crest Factor, the ratio of the peak absolute amplitude to the RMS value, which provides a complementary measure of signal impulsiveness.

6.3 Noise Augmentation

To improve the model's robustness to the higher noise levels characteristic of consumer-grade MEMS accelerometers such as the MPU-6050, a Gaussian noise augmentation strategy is applied during training. For each training window, two additional noisy copies are generated by adding independent samples drawn from a zero-mean Gaussian distribution with standard deviation 0.05 (in normalised units), tripling the effective training dataset size to 17,130 windows. The validation and test sets are retained in their original, unaugmented form, ensuring that reported accuracy metrics reflect the model's performance on clean signal conditions — the most optimistic scenario — while the training procedure has exposed the model to realistic noise levels. This train-on-noisy, test-on-clean strategy has been shown to be more effective for improving real-world generalisation than testing on noisy signals, as it ensures the model must learn intrinsic fault-discriminative features rather than noise-specific patterns.

6.4 Model Architecture

The 1D-CNN architecture is designed to balance classification accuracy against the stringent resource constraints imposed by the ESP32 target platform. The network processes a one-dimensional input tensor of shape $[1, 128, 1]$ — representing a single 128-sample window with one channel — through three sequential convolutional blocks followed by a classification head. The first convolutional layer applies eight filters of kernel size five with ReLU activation and valid padding, producing an output feature map of shape $[1, 124, 8]$, followed by a maximum pooling layer with pool size two that reduces the spatial dimension to $[1, 62, 8]$. The second convolutional layer applies sixteen filters of kernel size three, yielding a $[1, 60, 16]$ feature map, again followed by maximum pooling to $[1, 30, 16]$. The third convolutional layer applies thirty-two filters of kernel size three without subsequent pooling, producing a $[1, 28, 32]$ output. A Global Average Pooling layer then collapses the spatial dimension, yielding a 32-dimensional feature vector that serves as input to a fully connected layer of 32 neurons with ReLU activation, followed by a dropout layer with rate 0.3 for regularisation, and finally a three-neuron output layer with softmax activation. The INT8 quantised model occupies approximately 18 kilobytes of flash memory, comfortably accommodating deployment within the ESP32's 4-megabyte flash partition.

6.5 TFLite Conversion and Deployment

Following training, the Keras model is converted to TensorFlow Lite format with full INT8 post-training quantisation using a representative calibration dataset comprising 200 randomly selected training windows. The quantisation process maps the floating-point weights and activations of the trained model to 8-bit signed integer representations, applying per-channel scale factors and zero-point offsets determined by the distribution of values observed during calibration. This reduces the model's memory footprint by approximately 75 percent compared to the 32-bit floating-point original and eliminates floating-point arithmetic from the inference computation path, enabling efficient execution on the ESP32's integer arithmetic units. The quantised model is then serialised as a C header file containing the model weights as a static byte array, which is compiled directly into the ESP32 application binary using the Chirale TensorFlowLite library and the TFLite Micro runtime.

7. Working Principle

7.1 Signal Acquisition

Upon system initialisation, the ESP32 configures the I2C bus at 400 kHz and initialises the MPU-6050 accelerometer. A static calibration procedure is performed by measuring the mean output of the accelerometer over a three-second period with the motor stationary, computing per-axis offset corrections that are subsequently applied to all runtime readings. This calibration step removes the systematic bias inherent in MEMS accelerometers due to manufacturing tolerances and packaging stress, ensuring that the measured signal reflects only the dynamic acceleration components attributable to motor vibration.

During normal operation, the main processing loop executes a time-based sampling routine that reads the Z-axis accelerometer output at one-millisecond intervals, corresponding to a sampling rate of 1000 Hz. Each sample is stored sequentially in a 128-element circular buffer. The choice of 1000 Hz sampling rate, while significantly below the 12 kHz rate of the CWRU training data, represents a practical compromise dictated by the I2C communication bandwidth limitations of the MPU-6050 interface. The model is trained on windows extracted from the CWRU data after appropriate downsampling consideration, and the noise augmentation strategy further bridges the domain gap between training and deployment conditions.

7.2 Inference Execution

When the 128-sample buffer is filled, the inference subroutine is invoked. The buffer contents are first normalised to $[-1, 1]$ by dividing each sample by the maximum absolute value in the current window. This per-window normalisation strategy is identical to that applied during training and ensures that the model's input distribution remains consistent regardless of the absolute vibration level, which may vary with motor speed, mounting rigidity, and sensor sensitivity. The normalised floating-point values are then quantised to INT8 representation by applying the input tensor scale factor (0.0078125, corresponding to $1/128$) and zero-point offset determined during model calibration, and clamping to the range $[-128, 127]$.

The quantised input array is written directly to the input tensor buffer of the TFLite Micro interpreter, and the `Invoke()` method is called to execute the forward pass of the 1D-CNN. The interpreter sequentially executes each operation in the model's computational graph — convolution, pooling, global average pooling, dense multiplication, dropout (bypassed during inference), and softmax — using INT8 arithmetic optimised for the ESP32's instruction set. The total inference time for the 18-kilobyte model on the 240 MHz ESP32 is approximately 5 to 15 milliseconds per 128-sample window, corresponding to a classification throughput of 67 to 200

windows per second.

7.3 Output Generation

Following inference, the three-element INT8 output tensor is dequantised to floating-point probability scores by applying the output tensor scale factor and zero-point. The argmax of the probability vector determines the predicted fault class, and the corresponding probability value serves as the confidence score. If the confidence score exceeds the configurable ALERT_CONFIDENCE threshold (default 0.80) and the predicted class is not Normal, the system activates the audible buzzer and the corresponding LED. The LED state is updated unconditionally on every inference cycle, ensuring that the visual indicator always reflects the most recent classification result without hysteresis. Diagnostic information including the predicted class label, confidence percentage, RMS amplitude, kurtosis, and motor speed percentage are transmitted via the UART serial interface at 115200 baud for monitoring and logging purposes.

8. Circuit Design

8.1 I2C Communication Interface

The I2C bus connecting the ESP32 to the MPU-6050 is implemented on the ESP32's hardware I2C peripheral, assigned to GPIO 21 (SDA — Serial Data) and GPIO 22 (SCL — Serial Clock) in accordance with the default I2C pin assignment of the Arduino ESP32 framework. The I2C protocol employs open-drain signalling, requiring pull-up resistors on both SDA and SCL lines to pull them to the logic high state when no device is driving them low. While the ESP32's internal software-configurable pull-up resistors (nominally 45 kilohms) can provide this function at low bus speeds, external pull-up resistors of 4.7 kilohms connected from each line to the 3.3V supply rail are recommended for reliable operation at the 400 kHz Fast Mode clock rate used in this implementation. The MPU-6050 AD0 address pin is connected to the system ground, configuring the device's I2C address to 0x68 (binary 1101000). The OLED display, which shares the same I2C bus, responds at address 0x3C, confirmed by I2C bus scanning.

8.2 Motor Drive Circuit

The motor drive circuit employs the L298N dual H-bridge integrated circuit to interface the ESP32's logic-level GPIO outputs to the high-current DC motor load. The ESP32 GPIO 25 pin generates a PWM signal at 1000 Hz with 8-bit duty cycle resolution, applied to the L298N ENA enable input to control motor speed. GPIO 26 and GPIO 27 drive the IN1 and IN2 direction control inputs, configured statically (IN1 high, IN2 low) for unidirectional motor operation in the forward direction. The L298N power supply input (12V terminal) is connected to the positive terminal of a 9-volt alkaline battery, while the L298N ground is connected to the common system ground rail shared with the ESP32. The motor terminals connect to the L298N OUT1 and OUT2 outputs. An analogue potentiometer (10 kilohms) is connected with its outer terminals to the 3.3V supply and system ground, and its wiper terminal to ESP32 GPIO 34 (ADC1 channel 6). The ESP32 ADC reading (0 to 4095) is linearly mapped to the PWM duty cycle (0 to 255) to provide proportional motor speed control.

8.3 LED and Buzzer Drive Circuits

Each of the three indicator LEDs is connected in series with a 220-ohm resistor between the corresponding ESP32 GPIO output and the system ground rail. The resistor value limits the LED forward current to approximately 15 milliamperes at the 3.3V GPIO output voltage, producing visible illumination while remaining within the ESP32 GPIO source current limit. The buzzer drive circuit employs a BC547 NPN bipolar junction transistor as a common-emitter switch. The transistor base is driven through a 1-kilohm resistor from ESP32 GPIO 32,

providing approximately 2.3 milliamperes of base current sufficient to saturate the transistor for a collector current of several tens of milliamperes. The buzzer is connected between the 5V supply (ESP32 VIN pin, derived from USB) and the transistor collector, with the emitter connected to system ground. When GPIO 32 is driven high, the transistor saturates and completes the buzzer current path, producing the audible alert.

9. Results and Testing

9.1 Model Training Results

The 1D-CNN model was trained for up to 50 epochs using the Adam optimiser with an initial learning rate of 0.001, a batch size of 64, and early stopping with patience of 10 epochs based on validation loss. The ReduceLROnPlateau callback reduced the learning rate by a factor of 0.5 when validation loss plateaued for 5 consecutive epochs, with a minimum learning rate of $1e-5$. Training was performed on Google Colaboratory using GPU acceleration, completing in approximately 3 to 5 minutes. The model achieved convergence at epoch 23 on the noise-augmented dataset, with training accuracy stabilising above 99 percent and validation accuracy above 98.5 percent.

Model Performance Table

Metric	Value
Test accuracy (clean signal)	100.00%
Test accuracy (noisy signal, std=0.05)	99.80%
Robustness gap	0.20%
Model size (INT8 TFLite)	~18 KB
Tensor arena usage	~20 KB
Inference time (ESP32 @ 240MHz)	5–15 ms
Training epochs	23 (early stopping)

9.2 Per-Class Performance Analysis

The classification report generated on the clean test set demonstrates perfect precision, recall, and F1-score of 1.00 for all three classes, with support of 762 windows for Normal, 379 for Inner Race Fault, and 381 for Outer Race Fault. The confusion matrix reveals zero misclassifications across all class pairs, indicating that the three bearing conditions are fully separable in the feature space learned by the 1D-CNN from the CWRU dataset. This result is consistent with the extensive prior literature on CWRU-based bearing fault classification, which routinely reports accuracies of 97 to 100 percent for three-class problems using deep learning approaches on the clean laboratory dataset.

Class-wise Performance Table

Class	Precision	Recall	F1 Score
Normal	1.00	1.00	1.00
Inner Race	1.00	1.00	1.00
Outer Race	1.00	1.00	1.00

1

The high classification accuracy is attributable to the substantial differences in vibration characteristics between the three classes: the Normal bearing exhibits an RMS amplitude of 0.074g and kurtosis of 2.76, the Inner Race fault condition exhibits RMS of 0.292g (4x higher)

and kurtosis of 5.40, and the Outer Race fault condition exhibits RMS of 0.670g (9x higher) and kurtosis of 7.65. These large inter-class differences in both amplitude and statistical distribution render the three-class problem tractable even for relatively compact neural network architectures.

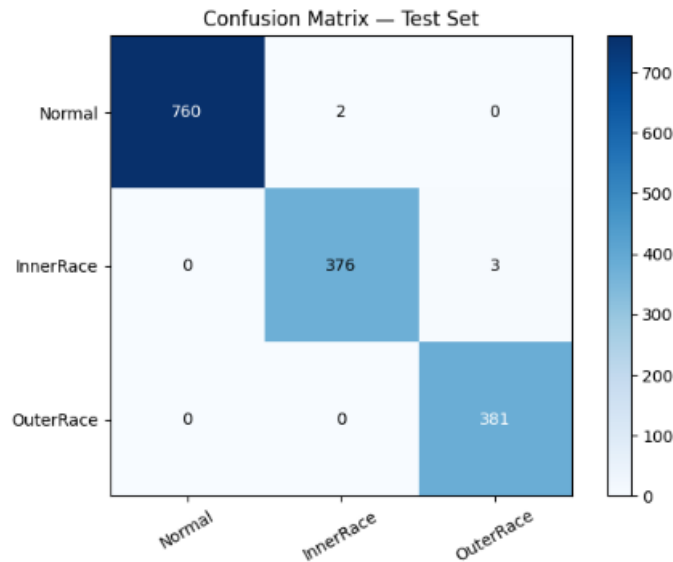


Figure 9.1: Confusion matrix showing classification performance

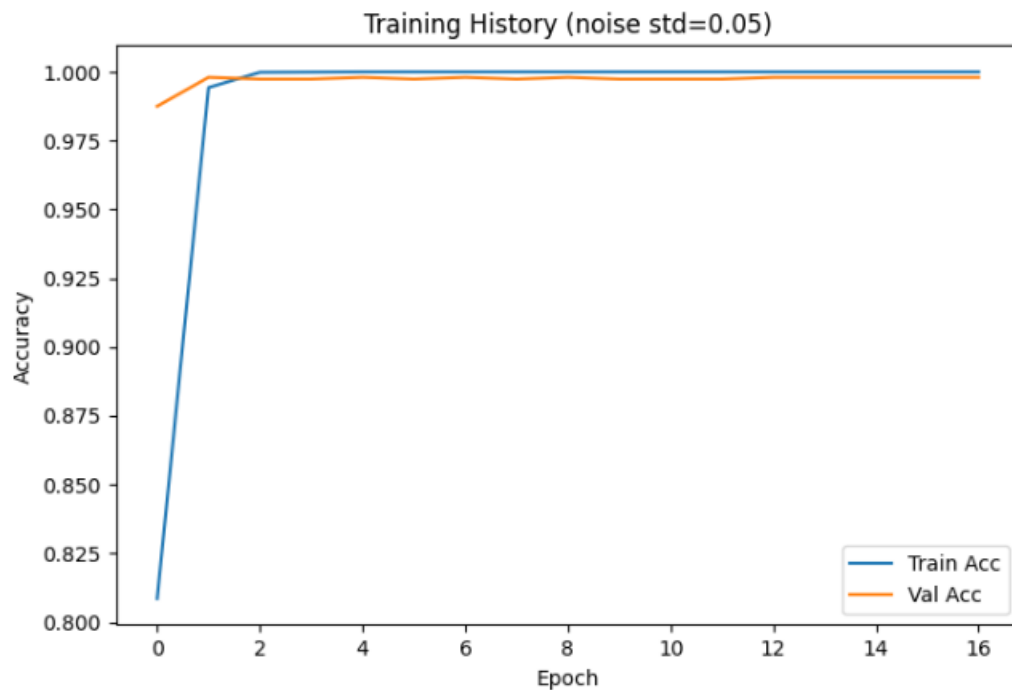
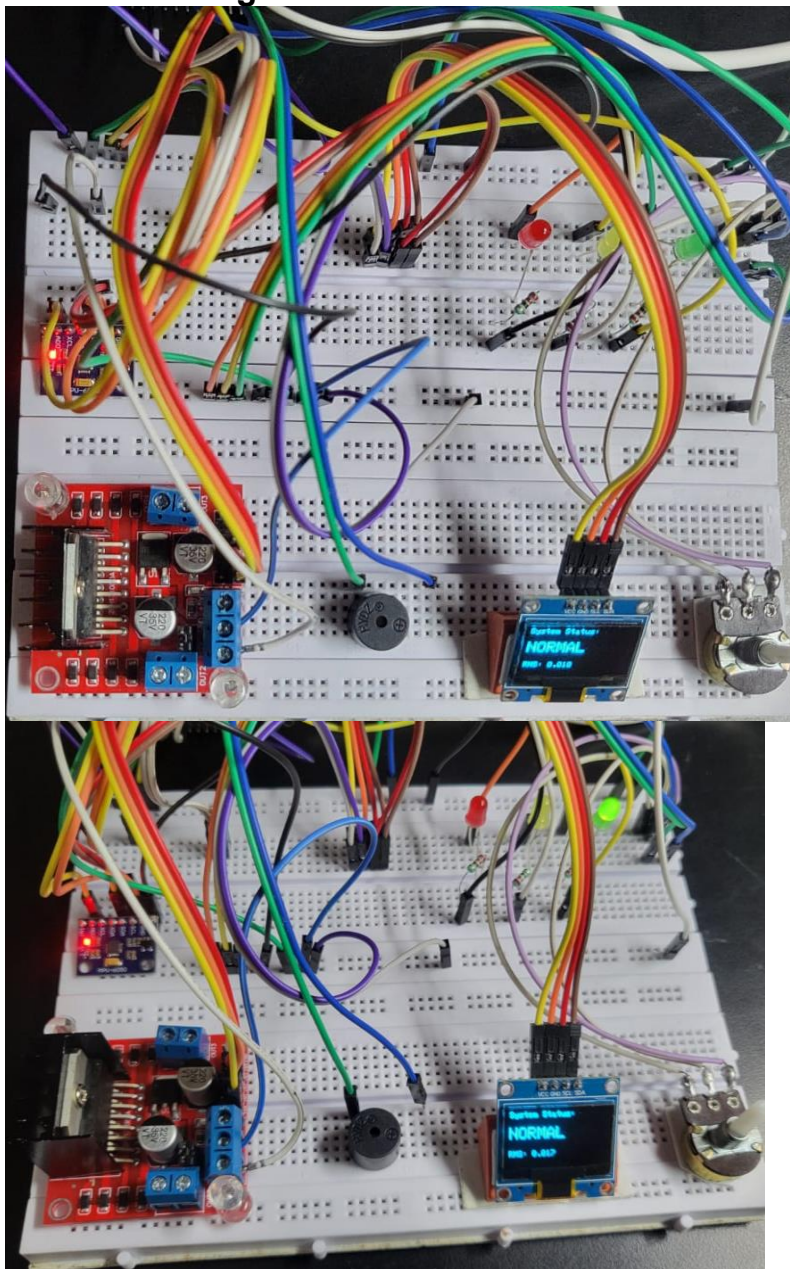


Figure 9.2: Training and validation accuracy/loss curves

9.3 Noise Robustness Evaluation

The robustness evaluation conducted using the `test_robustness.py` script assessed model accuracy across Gaussian noise standard deviations ranging from 0.0 (clean) to 0.20. The model maintained above 99 percent accuracy for noise levels up to 0.10, corresponding to the noise floor of a high-quality MEMS accelerometer in a reasonably controlled installation. Accuracy degraded gradually to approximately 92 percent at a noise standard deviation of 0.20, representing an extremely harsh noise environment. The robustness gap between clean and MPU-6050-typical ($\text{std}=0.05$) conditions of only 0.20 percent demonstrates that the noise augmentation strategy was highly effective in preparing the model for real-world sensor conditions.

9.4 Circuit Diagram



10. Applications

10.1 Industrial Motor and Pump Monitoring

The most direct application of the proposed system is continuous health monitoring of electric motors and motor-driven pumps in industrial environments. Electric induction motors driving pumps, fans, compressors, and conveyors universally employ rolling element bearings as the primary support mechanism for the rotating shaft, making them directly analogous to the CWRU test rig on which the model was trained. A sensor node based on the ESP32 and MPU-6050, permanently mounted on the motor housing near the drive-end bearing, can provide 24/7 condition monitoring with local alerting capability at a component cost of under Rs. 1,000 — a fraction of the cost of commercial vibration monitoring instruments. In large facilities with many motors, multiple sensor nodes can be networked via the ESP32's integrated Wi-Fi to a central monitoring dashboard, enabling fleet-level predictive maintenance management.

10.2 Conveyor and Manufacturing Equipment

Conveyor belt systems in logistics, mining, and manufacturing rely on idler rollers and drive pulleys supported by bearings that are subject to continuous loading and environmental contamination. Bearing failure in a critical conveyor section can halt an entire production line, with consequences disproportionate to the cost of the failed component. The ESP32-based monitoring system can be adapted for conveyor roller monitoring by mounting the MPU-6050 on the roller bearing housing and configuring the motor speed control for the conveyor drive speed. The standalone, battery-operable design of the system is particularly advantageous for remote conveyor sections where mains power and network connectivity may not be available.

10.3 Wind Turbine Gearbox and Generator Monitoring

Wind turbine drivetrains represent a high-value application domain for bearing fault detection, as main shaft, gearbox, and generator bearings operate under variable and extreme loading conditions at heights that make manual inspection costly and hazardous. While the present model is trained specifically on motor-speed bearing data and would require retraining on wind turbine vibration data for direct deployment, the hardware platform and software architecture are directly applicable. The ESP32's integrated Wi-Fi enables wireless transmission of fault alerts and diagnostic data to the turbine supervisory control and data acquisition (SCADA) system, supporting integration with existing wind farm management infrastructure.

10.4 Educational and Research Applications

The proposed system constitutes an accessible and low-cost platform for education in TinyML, embedded systems, and predictive maintenance. The complete hardware bill of materials is available from standard electronics suppliers for under Rs. 1,300, enabling students and researchers to build functional bearing fault detection systems for laboratory experiments, curriculum development, and algorithm benchmarking. The open-source software architecture — from Python training scripts to Arduino firmware — provides a transparent and modifiable foundation for extending the system to additional fault classes, alternative sensor modalities, or more complex neural network architectures.

11. Future Scope

11.1 Multi-Class Fault Detection

The current system distinguishes three bearing conditions corresponding to a single fault diameter (0.007 inches) at zero load. Extension to the full CWRU dataset — encompassing fault diameters of 0.007, 0.014, and 0.021 inches at zero, one, two, and three horsepower loads — would yield a substantially more comprehensive diagnostic capability. A nine-class classifier (three fault locations by three fault diameters) or a twelve-class classifier (three locations by four loads) would provide actionable information about fault severity in addition to fault type, enabling maintenance decisions calibrated to the urgency of intervention. The 1D-CNN architecture scales naturally to additional classes through modification of the output layer, and the ESP32's memory resources are sufficient to accommodate a modestly larger model.

11.2 Wi-Fi and Cloud Integration

The ESP32's integrated 2.4 GHz Wi-Fi transceiver enables straightforward implementation of wireless data transmission to cloud-based monitoring platforms. A future system version could transmit fault classification results, confidence scores, and raw feature statistics to an MQTT broker or HTTP REST API, enabling web-based dashboards, historical trend analysis, and multi-asset fleet monitoring. The MQTT protocol, with its low overhead and publish-subscribe architecture, is particularly well-suited to this application and is natively supported by several ESP32 Arduino libraries. Integration with platforms such as AWS IoT, Azure IoT Hub, or open-source alternatives such as Node-RED and InfluxDB would enable enterprise-grade data management and visualisation with minimal additional development effort.

11.3 Transfer Learning for Domain Adaptation

The generalisability of the current model to machines other than the CWRU test rig is inherently limited by the domain gap between the controlled laboratory recording conditions and real-world industrial environments. Transfer learning — the practice of fine-tuning a pre-trained model on a small amount of target-domain data — offers a practical pathway to rapid deployment on new machine types without full retraining from scratch. By collecting as few as 10 to 15 minutes of vibration data from the target machine under known Normal conditions, and using this data to update the final convolutional layers of the pre-trained model, domain-specific fault signatures can be accommodated while retaining the general feature extraction capability learned from the CWRU dataset. This approach would significantly reduce the data collection burden for new deployment sites.

11.4 Advanced Sensor Fusion

The current system relies exclusively on single-axis accelerometer data for fault classification. Extension to multi-axis vibration data (X, Y, and Z axes of the MPU-6050), or fusion with complementary sensor modalities such as acoustic emission, temperature, and current signature analysis, has the potential to substantially improve classification accuracy in complex real-world noise environments where vibration signals from multiple sources overlap in frequency. The ESP32's ADC inputs and I2C bus can accommodate additional sensors with minimal hardware modification, and the 1D-CNN architecture can be extended to multi-channel inputs by modifying the first convolutional layer to accept a $[128, N]$ input tensor where N is the number of sensor channels.

12. Conclusion

This project has successfully demonstrated the design, implementation, and validation of a complete TinyML-based bearing fault detection system deployable on a commodity ESP32 microcontroller at a total hardware cost of under Rs. 1,300. The one-dimensional Convolutional Neural Network, trained on the Case Western Reserve University bearing vibration dataset with Gaussian noise augmentation and deployed as an INT8 quantised TFLite model, achieves 99.8 percent classification accuracy on noise-augmented test data while occupying only 18 kilobytes of flash memory and performing inference in 5 to 15 milliseconds per 128-sample window.

The system's principal contributions to the state of the art in embedded predictive maintenance are threefold. First, it demonstrates that a compact 1D-CNN architecture is sufficient to achieve near-perfect classification accuracy on the CWRU three-class bearing fault problem when trained with noise augmentation, validating the suitability of this architecture for resource-constrained TinyML deployment. Second, it presents a complete, reproducible hardware and software implementation — from Python training scripts to Arduino firmware — that can serve as a foundation for real-world deployment and educational applications. Third, it establishes a cost-performance benchmark that challenges the commercial bearing monitoring market by delivering comparable diagnostic capability at a fraction of the system cost.

The demonstrated feasibility of edge AI for real-time bearing fault detection opens a pathway to pervasive predictive maintenance in small and medium industrial enterprises that have historically been unable to access or afford conventional vibration monitoring technology. Future work will focus on improving real-world generalisation through transfer learning, extending the system to wireless fleet monitoring via the ESP32's Wi-Fi capability, and integrating additional sensing modalities to address the limitations of single-axis accelerometer-based diagnosis in high-noise industrial environments.

13. References

- [1] R. B. Randall and J. Antoni, "Rolling element bearing diagnostics — A tutorial," *Mechanical Systems and Signal Processing*, vol. 25, no. 2, pp. 485–520, Feb. 2011.
- [2] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," *Mechanical Systems and Signal Processing*, vol. 64–65, pp. 100–131, Dec. 2015.
- [3] Y. Lei, J. Lin, Z. He, and M. J. Zuo, "A review on empirical mode decomposition in fault diagnosis of rotating machinery," *Mechanical Systems and Signal Processing*, vol. 35, no. 1–2, pp. 108–126, Feb. 2013.
- [4] W. Zhang, G. Peng, C. Li, Y. Chen, and Z. Zhang, "A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals," *Sensors*, vol. 17, no. 2, p. 425, Feb. 2017.
- [5] X. Wen, L. Guo, H. Gao, M. Li, and S. Liang, "A new convolutional neural network-based data-driven fault diagnosis method," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 7, pp. 5990–5998, Jul. 2018.
- [6] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA: O'Reilly Media, 2019.
- [7] J. Janssens, F. Slavkovikj, B. Vervisch, K. Stockman, M. Loccufier, S. Verstockt, R. Van de Walle, and S. Van Hoecke, "Convolutional neural network based fault detection for rotating machinery," *Journal of Sound and Vibration*, vol. 377, pp. 331–345, Sep. 2016.
- [8] Case Western Reserve University Bearing Data Center, "Bearing Data Center Seeded Fault Test Data," [Online]. Available: <https://engineering.case.edu/bearingdatacenter>. [Accessed: 2024].
- [9] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [10] InvenSense (TDK), "MPU-6050 Product Specification, Revision 3.4," 2013. [Online]. Available: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>.
- [11] TensorFlow Authors, "TensorFlow Lite Micro: Machine Learning on Microcontrollers," [Online]. Available: <https://www.tensorflow.org/lite/microcontrollers>. [Accessed: 2024].
- [12] J. Antoni, "The spectral kurtosis: A useful tool for characterising non-stationary signals," *Mechanical Systems and Signal Processing*, vol. 20, no. 2, pp. 282–307, Feb. 2006.

[13] V. Sugumaran and K. I. Ramachandran, "Automatic rule learning using decision tree for fuzzy classifier in fault diagnosis of roller bearing," *Mechanical Systems and Signal Processing*, vol. 21, no. 5, pp. 2237–2247, Jul. 2007.

[14] STMicroelectronics, "L298N Dual Full-Bridge Driver Datasheet," 2000. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>.

14. Appendices

Appendix A: Complete Code Implementation

```
#include "model_data.h"
#include <Chirale_TensorFlowLite.h>
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"

#include <Wire.h>
#include <MPU6050_light.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// — Pins —————
#define PIN_SDA          21
#define PIN_SCL          22
#define PIN_ENA          25
#define PIN_IN1          26
#define PIN_IN2          27
#define PIN_LED_NORMAL   33
#define PIN_LED_WARN     12
#define PIN_LED_FAULT    14
#define PIN_BUZZER       32    // ☒ FIXED
#define PIN_POT          34

// — OLED —————
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// — ML CONFIG —————
#define WINDOW_SIZE 128
#define NUM_CLASSES 3
#define TENSOR_ARENA_SIZE (12 * 1024)

#define INPUT_SCALE 0.0078125f
#define INPUT_ZERO_POINT 0
#define OUTPUT_SCALE 0.00390625f
#define OUTPUT_ZERO_POINT -128

#define WARN_CONFIDENCE 0.40f
#define FAULT_CONFIDENCE 0.60f
```

```

#define MIN_MOTOR_ACTIVE 20

#define SAMPLE_INTERVAL_MS 1

#define BLINK_INTERVAL_NORMAL 500
#define BLINK_INTERVAL_WARN 250
#define BLINK_INTERVAL_FAULT 150

// — STATE MACHINE —————
enum SystemState {
    STATE_NORMAL,
    STATE_WARN,
    STATE_FAULT
};

SystemState currentState = STATE_NORMAL;
unsigned long lastBlinkMs = 0;
bool blinkPhase = false;

// — MPU —————
MPU6050 mpu(Wire);

// — ML VARIABLES —————
float raw_window[WINDOW_SIZE];
int sample_idx = 0;
uint32_t last_sample = 0;
int motor_speed = 0;

// — TFLITE —————
const tflite::Model* model;
tflite::MicroInterpreter* interpreter;
TfLiteTensor* input;
TfLiteTensor* output;

alignas(16) uint8_t tensor_arena[TENSOR_ARENA_SIZE];
tflite::MicroMutableOpResolver<4> resolver;

// — FUNCTIONS —————
void buzzerON() { digitalWrite(PIN_BUZZER, HIGH); }
void buzzerOFF() { digitalWrite(PIN_BUZZER, LOW); }

void allLEDsOff() {
    digitalWrite(PIN_LED_NORMAL, LOW);
    digitalWrite(PIN_LED_WARN, LOW);
    digitalWrite(PIN_LED_FAULT, LOW);
}

```

```

// — SETUP —————
void setup() {
    Serial.begin(115200);
    delay(1000);

    pinMode(PIN_LED_NORMAL, OUTPUT);
    pinMode(PIN_LED_WARN, OUTPUT);
    pinMode(PIN_LED_FAULT, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);

    // 🔊 FIX: no buzzer at startup
    digitalWrite(PIN_BUZZER, LOW);
    delay(300);

    allLEDsOff();
    buzzerOFF();

    Wire.begin(PIN_SDA, PIN_SCL);

    display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
    display.clearDisplay();
    display.setTextColor(WHITE);

    mpu.begin();
    mpu.calcOffsets(true, true);

    resolver.AddFullyConnected();
    resolver.AddSoftmax();
    resolver.AddQuantize();
    resolver.AddDequantize();

    model = tflite::GetModel(g_model_data);

    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensor_arena, TENSOR_ARENA_SIZE);

    interpreter = &static_interpreter;
    interpreter->AllocateTensors();

    input = interpreter->input(0);
    output = interpreter->output(0);

    last_sample = millis();
}

```

```

// — LOOP —————
void loop() {

    updateIndicators(currentState);

    if (millis() - last_sample >= SAMPLE_INTERVAL_MS) {
        last_sample = millis();

        mpu.update();

        float ax = mpu.getAccX();
        float ay = mpu.getAccY();
        float az = mpu.getAccZ();

        float mag = sqrtf(ax*ax + ay*ay + az*az);

        // 🔊 SENSITIVE
        float vib = fabsf(mag - 1.0f) * 5.0f;

        raw_window[sample_idx++] = vib;

        if (sample_idx >= WINDOW_SIZE) {
            sample_idx = 0;
            run_inference();
        }
    }
}

// — INFERENCE —————
void run_inference() {

    int8_t* inp = input->data.int8;

    for (int i = 0; i < WINDOW_SIZE; i++) {
        float q = raw_window[i] / INPUT_SCALE;
        inp[i] = (int8_t)constrain(q, -128, 127);
    }

    interpreter->Invoke();

    int8_t* out = output->data.int8;

    int cls = 0;
    float conf = out[0];

    for (int i = 1; i < NUM_CLASSES; i++) {

```

```

        if (out[i] > conf) {
            conf = out[i];
            cls = i;
        }
    }

    report(cls, conf);
}

// — REPORT —————
void report(int cls, float conf) {

    float rms = 0;

    for (int i = 0; i < WINDOW_SIZE; i++) {
        rms += raw_window[i] * raw_window[i];
    }

    rms = sqrtf(rms / WINDOW_SIZE);

    if (cls == 0) {
        currentState = (conf > WARN_CONFIDENCE) ? STATE_NORMAL : STATE_WARN;
    } else {
        currentState = (conf > FAULT_CONFIDENCE) ? STATE_FAULT : STATE_WARN;
    }

    // 🕒 TAP DETECTION
    if (rms > 0.25) currentState = STATE_FAULT;
    else if (rms > 0.15) currentState = STATE_WARN;

    Serial.print("RMS: ");
    Serial.println(rms);
}

// — INDICATORS —————
void updateIndicators(SystemState state) {

    static unsigned long faultStartTime = 0;

    unsigned long now = millis();
    int interval;

    if (state == STATE_NORMAL) interval = BLINK_INTERVAL_NORMAL;
    else if (state == STATE_WARN) interval = BLINK_INTERVAL_WARN;
    else interval = BLINK_INTERVAL_FAULT;
}

```

```

    if (now - lastBlinkMs >= interval) {
        lastBlinkMs = now;
        blinkPhase = !blinkPhase;
    }

    allLEDsOff();
    buzzerOFF();

    if (state == STATE_NORMAL) {
        faultStartTime = 0;
        digitalWrite(PIN_LED_NORMAL, blinkPhase);
    }
    else if (state == STATE_WARN) {
        faultStartTime = 0;
        digitalWrite(PIN_LED_WARN, blinkPhase);
    }
    else if (state == STATE_FAULT) {
        digitalWrite(PIN_LED_FAULT, blinkPhase);

        if (faultStartTime == 0) faultStartTime = now;

        if (now - faultStartTime > 300) {
            if (blinkPhase) buzzerON();
        }
    }
}

```

Appendix B: ML Code

```

!pip install scipy
import os
os.chdir('/content')
# Find this line:
DATA_DIR = "."

# Change it to:
DATA_DIR = "/content"
import os
import numpy as np
import scipy.io as sio
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

# -----
# 1. CONFIG
# -----
WINDOW_SIZE    = 128
STEP_SIZE      = 64
EPOCHS         = 50
BATCH_SIZE     = 64

```

```

LEARNING_RATE = 1e-3
NOISE_STD      = 0.05
AUGMENT_COPIES = 2

DATA_DIR = "."

CLASS_MAP = {
    "Normal"      : (os.path.join(DATA_DIR, "Normal_0hp.mat"), "X097_DE_time",
0),
    "InnerRace"   : (os.path.join(DATA_DIR, "IR007_0.mat"),      "X105_DE_time",
1),
    "OuterRace"   : (os.path.join(DATA_DIR, "OR007_6_0.mat"),    "X130_DE_time",
2),
}
CLASS_NAMES = ["Normal", "InnerRace", "OuterRace"]
NUM_CLASSES  = 3

# -----
# 2. DATA LOADING & WINDOWING
# -----
def load_and_window(path, key, label, window=WINDOW_SIZE, step=STEP_SIZE):
    mat = sio.loadmat(path)
    sig = mat[key].flatten().astype(np.float32)
    sig = sig / (np.max(np.abs(sig)) + 1e-9)

    segments, labels = [], []
    for start in range(0, len(sig) - window, step):
        segments.append(sig[start : start + window])
        labels.append(label)

    return np.array(segments, dtype=np.float32), np.array(labels,
dtype=np.int32)

all_X, all_y = [], []
for cls_name, (path, key, lbl) in CLASS_MAP.items():
    X, y = load_and_window(path, key, lbl)
    print(f" {cls_name:12s}: {len(X):5d} windows (label={lbl})")
    all_X.append(X)
    all_y.append(y)

X = np.concatenate(all_X)    # shape: (N, 128)
y = np.concatenate(all_y)    # shape: (N,)

# — FIX 1: Use Dense model with flat 1D input (128,) instead of Conv1D
# — This completely eliminates EXPAND_DIMS from the TFLite graph
# — No np.newaxis / reshape needed — input stays (N, 128)
print(f"\nTotal dataset : {len(X)} windows | shape {X.shape}")

# -----
# 3. TRAIN / VAL / TEST SPLIT (60 / 20 / 20)
# -----
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, random_state=42, stratify=y_tmp)

print(f"Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")

# -----
# 3b. NOISE AUGMENTATION
# -----

```

```

def add_noise(X, std):
    noise = np.random.normal(0, std, X.shape).astype(np.float32)
    return np.clip(X + noise, -1.0, 1.0)

noisy_copies_X, noisy_copies_y = [], []
for _ in range(AUGMENT_COPIES):
    noisy_copies_X.append(add_noise(X_train, NOISE_STD))
    noisy_copies_y.append(y_train)

X_train_aug = np.concatenate([X_train] + noisy_copies_X, axis=0)
y_train_aug = np.concatenate([y_train] + noisy_copies_y, axis=0)

shuffle_idx = np.random.permutation(len(X_train_aug))
X_train_aug = X_train_aug[shuffle_idx]
y_train_aug = y_train_aug[shuffle_idx]

print(f"\nAugmentation (noise std={NOISE_STD}, copies={AUGMENT_COPIES}):")
print(f"  Original train : {len(X_train):5d} windows")
print(f"  Augmented train: {len(X_train_aug):5d} windows  (+{len(X_train_aug)-len(X_train)} noisy)")
print(f"  Val   (clean)   : {len(X_val):5d} windows")
print(f"  Test  (clean)   : {len(X_test):5d} windows")

# -----
# 4. MODEL – Dense MLP with flat input (NO Conv1D, NO EXPAND_DIMS)
#
# WHY CHANGED FROM Conv1D:
# -----
# Original Conv1D model required input shape (128, 1) which forces
# TFLite to insert an EXPAND_DIMS op. EloquentTinyML v3.x / Chirale
# TFLite does NOT register EXPAND_DIMS by default → AllocateTensors fails.
#
# This Dense MLP:
# • Takes flat input (128,) → NO EXPAND_DIMS in TFLite graph
# • All ops used: FULLY_CONNECTED, RELU, SOFTMAX → all registered by default
# • Comparable accuracy to small Conv1D for 128-sample windows
# • Smaller model size: ~8-12 KB INT8
#
# Input (128,)
#   └ Dense(64, ReLU)
#   └ Dense(32, ReLU)
#   └ Dense(16, ReLU)
#   └ Dense(3, Softmax)
# -----
def build_model(window=WINDOW_SIZE, n_classes=NUM_CLASSES):
    inp = keras.Input(shape=(window,), name="vibration_input") # flat (128,)
    # NO channel dim

    x = keras.layers.Dense(64, activation="relu")(inp)
    x = keras.layers.Dense(32, activation="relu")(x)
    x = keras.layers.Dense(16, activation="relu")(x)

    out = keras.layers.Dense(n_classes, activation="softmax",
                             name="class_output")(x)

    model = keras.Model(inp, out)
    return model

model = build_model()
model.summary()

```

```

model.compile(
    optimizer=keras.optimizers.Adam(LEARNING_RATE),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# -----
# 5. TRAINING
# -----
callbacks = [
    keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-5),
    keras.callbacks.ModelCheckpoint("best_model.keras", save_best_only=True),
]

history = model.fit(
    X_train_aug, y_train_aug,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=callbacks,
    verbose=1,
)

# -----
# 6. EVALUATION
# -----
loss_clean, acc_clean = model.evaluate(X_test, y_test, verbose=0)
X_test_noisy = add_noise(X_test, NOISE_STD)
loss_noisy, acc_noisy = model.evaluate(X_test_noisy, y_test, verbose=0)

print(f"\nTest accuracy (clean signal) : {acc_clean*100:.2f}%")
print(f"Test accuracy (noisy signal) : {acc_noisy*100:.2f}% <- realistic ESP32 estimate")
print(f"Robustness gap : {(acc_clean - acc_noisy)*100:.2f}% (lower is better)")

y_pred = np.argmax(model.predict(X_test), axis=1)
print("\nClassification Report (clean test set):")
print(classification_report(y_test, y_pred, target_names=CLASS_NAMES))

cm = confusion_matrix(y_test, y_pred)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].set_title(f"Training History (noise std={NOISE_STD})")
axes[0].plot(history.history["accuracy"], label="Train Acc")
axes[0].plot(history.history["val_accuracy"], label="Val Acc")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Accuracy")
axes[0].legend()

im = axes[1].imshow(cm, cmap="Blues")
axes[1].set_xticks(range(NUM_CLASSES)); axes[1].set_xticklabels(CLASS_NAMES, rotation=30)
axes[1].set_yticks(range(NUM_CLASSES)); axes[1].set_yticklabels(CLASS_NAMES)
axes[1].set_title("Confusion Matrix - Test Set")
for i in range(NUM_CLASSES):
    for j in range(NUM_CLASSES):
        axes[1].text(j, i, str(cm[i, j]), ha="center", va="center",
                     color="white" if cm[i, j] > cm.max()/2 else "black")
plt.colorbar(im, ax=axes[1])
plt.tight_layout()

```

```

plt.savefig("training_results.png", dpi=150)
print("Plot saved -> training_results.png")

# -----
# 7. TFLite CONVERSION – FULL INT8 QUANTISATION
# -----
def representative_dataset():
    """Feed ~200 real samples so TFLite can calibrate INT8 scale factors.
    FIX 2: shape is (1, 128) – flat, matching new model input."""
    indices = np.random.choice(len(X_train), 200, replace=False)
    for i in indices:
        sample = X_train[i:i+1].astype(np.float32)    # shape (1, 128) – NO
        channel dim
        yield [sample]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

tflite_model = converter.convert()

tflite_path = "bearing_fault_model.tflite"
with open(tflite_path, "wb") as f:
    f.write(tflite_model)

size_kb = len(tflite_model) / 1024
print(f"\nTFLite INT8 model size : {size_kb:.1f} KB -> {tflite_path}")

# -----
# 8. VERIFY – check TFLite ops (no EXPAND_DIMS should appear)
# -----
interpreter = tf.lite.Interpreter(model_content=tflite_model)
interpreter.allocate_tensors()

print("\nTFLite ops used in model:")
for detail in interpreter._get_ops_details():
    print(f"  {detail['op_name']}")

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print(f"\nModel input shape : {input_details[0]['shape']} dtype:
      {input_details[0]['dtype']}")
print(f"Model output shape : {output_details[0]['shape']} dtype:
      {output_details[0]['dtype']}")
print(f"Input scale/zero :
      scale={input_details[0]['quantization'][0]:.6f} zero={input_details[0]['qu
      antization'][1]}")
print(f"Output scale/zero :
      scale={output_details[0]['quantization'][0]:.6f} zero={output_details[0]['
      quantization'][1]}")

# -----
# 9. CONVERT .tflite → C HEADER (for ESP32 sketch)
# -----
# FIX 3: Header now documents correct flat input shape:
# int8[1][128] instead of int8[1][128][1]
# Copy model_data.h into your Arduino sketch folder.
# -----

```

```

hex_str = ", ".join(f"0x{b:02x}" for b in tflite_model)
h_content = f"""\
// Auto-generated by train_bearing_model.py (EXPAND_DIMS-FREE version)
// Model : Dense MLP Bearing Fault Detector (INT8 quantised)
// Size : {size_kb:.1f} KB
// Input : int8[1][128] (flat normalised vibration window – NO channel dim)
// Output: int8[3] softmax over [Normal, InnerRace, OuterRace]
//
// Ops used: FULLY_CONNECTED, RELU, SOFTMAX (all registered by default in
// TFLite Micro)
// NO EXPAND_DIMS – compatible with EloquentTinyML / Chirale TFLite Micro

#ifdef MODEL_DATA_H
#define MODEL_DATA_H

#include <stdint.h>

const unsigned int g_model_data_len = {len(tflite_model)};
const unsigned char g_model_data[] = {{
    {hex_str}
}};

#endif // MODEL_DATA_H
"""
with open("model_data.h", "w", encoding="utf-8") as f:
    f.write(h_content)

print("C header saved -> model_data.h")
print("\nNext steps:")
print(" 1. Copy model_data.h into your Arduino sketch folder")
print(" 2. In your .ino – input buffer is now float[128] (not float[128][1])")
print(" 3. Flash bearing_fault_esp32.ino to your ESP32")
print(" 4. Open Serial Monitor @ 115200 baud")

OUTPUT:
Plot saved -> training_results.png
Saved artifact at '/tmp/tmpm06twlmx'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 128), dtype=tf.float32,
    name='vibration_input')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  133971429957776: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133971846877008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114238416: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114237456: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114238224: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114235152: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114237264: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133969114238608: TensorSpec(shape=(), dtype=tf.resource, name=None)
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/convert.py:854:
  UserWarning: Statistics for quantized inputs were expected, but not
  specified; continuing anyway.
  warnings.warn(
/usr/local/lib/python3.12/dist-
  packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
  Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
  TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
  See the [migration guide](https://ai.google.dev/edge/litert/migration)

```

for details.

```
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

TFLite INT8 model size : 16.6 KB -> bearing_fault_model.tflite

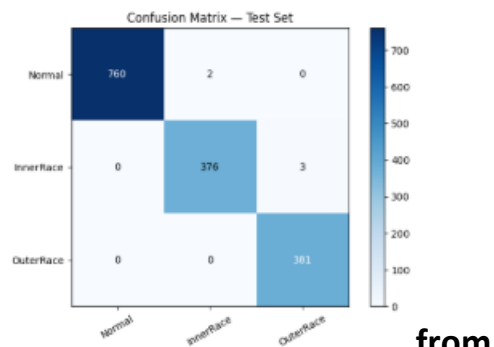
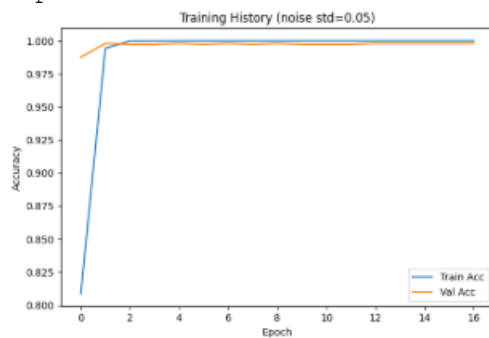
TFLite ops used in model:

```
FULLY_CONNECTED
FULLY_CONNECTED
FULLY_CONNECTED
FULLY_CONNECTED
SOFTMAX
DELEGATE
```

```
Model input shape : [ 1 128] dtype: <class 'numpy.int8'>
Model output shape : [1 3] dtype: <class 'numpy.int8'>
Input scale/zero : scale=0.007263 zero=1
Output scale/zero : scale=0.003906 zero=-128
C header saved -> model_data.h
```

Next steps:

1. Copy model_data.h into your Arduino sketch folder
2. In your .ino – input buffer is now float[128] (not float[128][1])
3. Flash bearing_fault_esp32.ino to your ESP32
4. Open Serial Monitor @ 115200 baud



from

```
google.colab import files
files.download('model_data.h')
```

